AWS Bastion Host Setup Guide

What is a Bastion Host?

- A **Bastion Host** (often called a "**jump server**" or "**jump box**") is a specialized, highly secured computer on a network specifically designed to withstand attacks. It acts as a single, fortified entry point—a secure bridge—allowing authorized users to access resources in a private network from an external, untrusted network like the internet.

How it Works?

- A typical cloud-based architecture (e.g., AWS, Azure, GCP) involves several layers:
  - **Public Subnet:** The Bastion Host lives here and has a public IP address.
  - **Private Subnet:** Sensitive resources live here with only private IP addresses; they have no direct internet access.
  - **The Flow:**
    - A user connects to the Bastion Host from the internet (usually restricted to their specific office/home IP).
    - The user authenticates with high-security measures like **Multi-Factor Authentication (MFA)** or **SSH key pairs**.
    - Once inside, the user initiates a second connection (a "jump") to the private server.

Core Function & Purpose:

- The primary goal of a bastion host is to **minimize the attack surface** of a private network.
  - **The Problem:** Sensitive servers (like databases or app backends) should never be directly accessible from the internet.
  - **The Solution:** Instead of exposing every private server, you only expose one: the Bastion Host. Administrators "jump" through this host to reach internal resources via SSH (Linux) or RDP (Windows).

Key Characteristics:

- **Hardened Configuration:** To reduce vulnerability, the host runs only the bare minimum services required for its task. All unnecessary software, ports, and protocols are removed or disabled.
- **Centralized Logging:** It provides a central chokepoint to log and audit every administrative session, capturing who logged in, when, and sometimes even the specific commands executed.
- **Network Isolation:** Security groups are configured so that private servers only accept connections from the Bastion Host's private IP.

Why use one?

- **Reduced Attack Surface**: Instead of opening 10 servers to the internet, you only open one.
- **Centralized Logging**: You can record exactly who logged in and what commands they ran in one place.
- **Stronger Auth**: You can enforce Multi-Factor Authentication (MFA) on the Bastion to protect the entire private backend.

Drawbacks & Modern Alternatives:

- While effective, bastion hosts have notable downsides:
  - **Single Point of Failure:** If the bastion is compromised, the attacker has a direct path to the internal network. If it goes down, administrators lose access entirely.
  - **Management Overhead:** They require regular patching, auditing, and maintenance.
  - **Modern Shifts:** Many organizations are moving toward **Zero Trust Network Access (ZTNA)** or cloud-native managed services like AWS Systems Manager Session Manager, Azure Bastion, or GCP Identity-Aware Proxy (IAP), which allow secure access without needing a public-facing server at all.

Provider	Service Name	Key Benefit
AWS	Systems Manager (SSM)	Access private EC2s via the AWS Console/CLI without opening port 22.
Azure	Azure Bastion	A fully managed PaaS that lets you RDP/SSH into VMs directly through your browser over SSL (port 443).
GCP	Identity-Aware Proxy (IAP)	Uses Google identities to grant SSH access without needing a public IP on the VM.

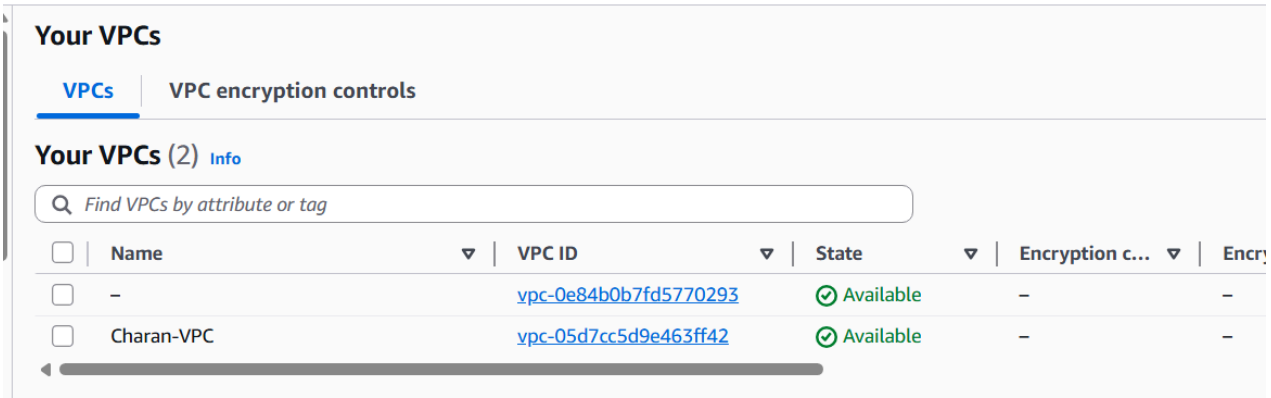
Lab: Creating a Bastion Host/Jump Server in AWS

- Setting up a bastion host in AWS involves creating two servers in different network layers: a **Bastion Server** in a public subnet and a **Private Server** in a private subnet.

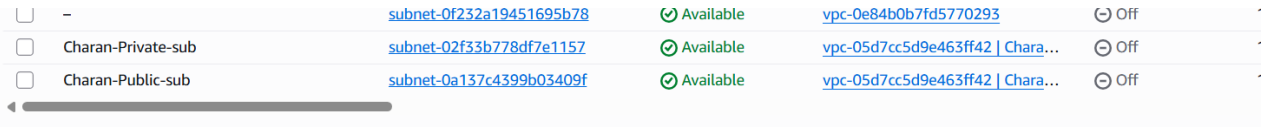
Step 1: Network Infrastructure (VPC & Subnets)

First, create a Virtual Private Cloud (VPC) to house your resources.

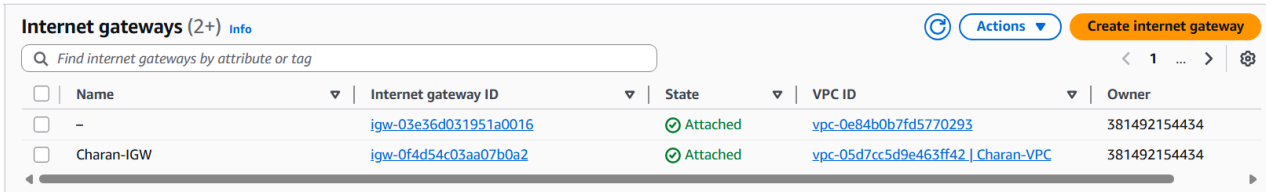
- **Create VPC**: Navigate to **VPC** > **Create VPC**.
  1. **CIDR Block**: `10.0.0.0/16`.



- **Create Subnets**:
  1. **Public Subnet**: `10.0.1.0/24` (for Bastion).
  2. **Private Subnet**: `10.0.2.0/24` (for Private Server).

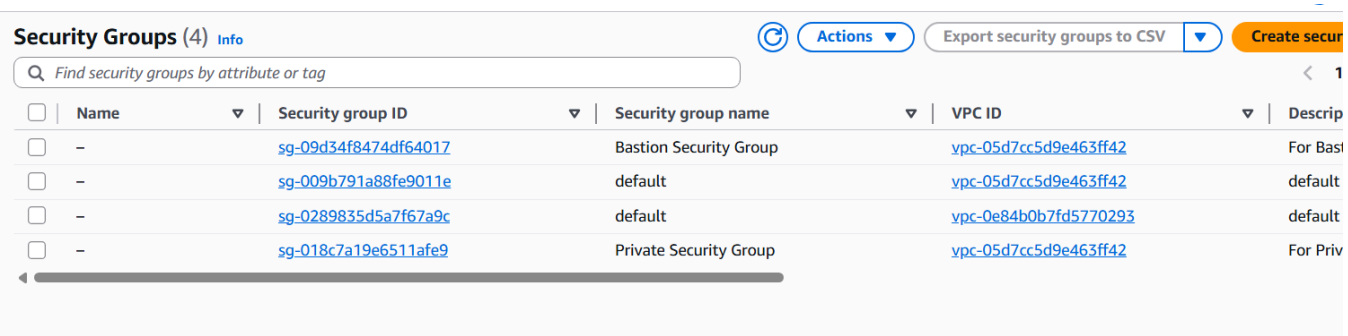


- 1. **Setup Internet Access**:
    1. Create an **Internet Gateway (IGW)** and attach it to your VPC.



Step 2: Security Groups

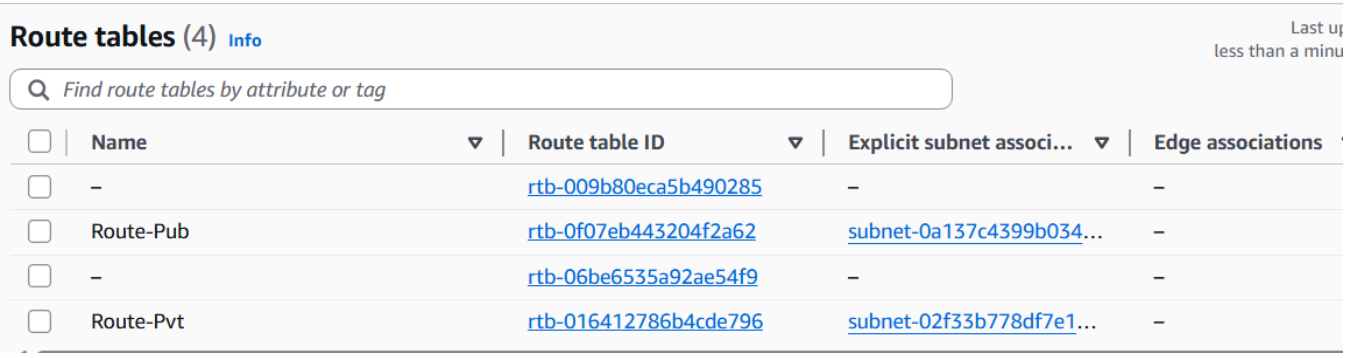
- Configure firewalls to control traffic between layers.
  1. **Bastion Security Group**:
     1. **Inbound**: Allow `SSH` (Port 22) from **My IP** only.
  2. **Private Security Group**:
     1. **Inbound**: Allow `SSH` (Port 22) **only** from the **Bastion Security Group ID**.



Step 3: Configure Routing

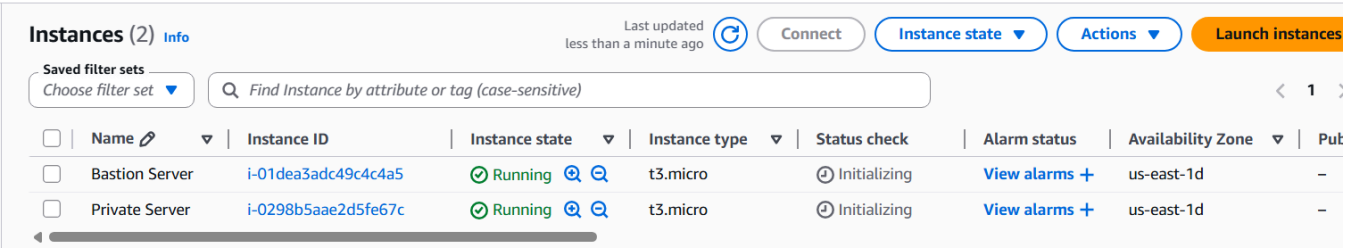
- Create and associate route tables to direct traffic.
  1. **Public:** Create a route table, add a route to the Internet Gateway (`0.0.0.0/0`), and associate it with the public subnet.
  2. **Private:** Create a separate route table and associate it with the private subnet (no direct internet

route).



Step 4: Launching the Servers

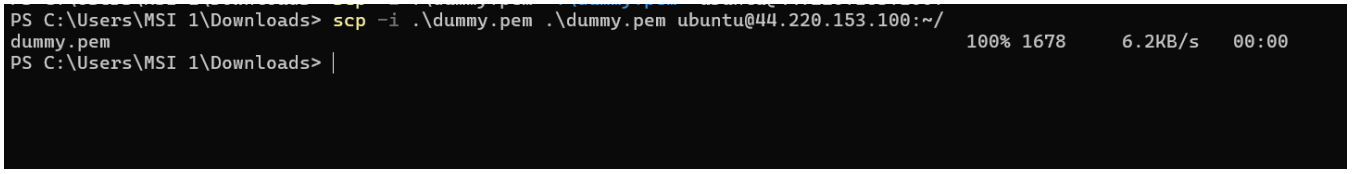
- Use the **EC2 Dashboard** to launch your instances.
    1. **Bastion Server (Public)**:
        1. **Subnet**: Select **Public Subnet**.
        2. **Auto-assign Public IP**: Enable.
        3. **Security Group**: Use the Bastion Security Group.
    2. **Private Server (Private)**:
        1. **Subnet**: Select **Private Subnet**.
        2. **Auto-assign Public IP**: Disable.
        3. **Security Group**: Use the Private Security Group.



Step 5: Copy the Private Key to the Bastion

- From your **local machine**, use `scp` (Secure Copy) to upload your `.pem` file to the Bastion's home directory.

```
# Run this on your local computer
scp -i "your-key.pem" your-key.pem ec2-user@BASTION_PUBLIC_IP:~/.
```



Step 6: Log into the Bastion Host

- Now, SSH into the Bastion as you normally would.

```
# Run this on your local computer
ssh -i "your-key.pem" ec2-user@BASTION_PUBLIC_IP
```

```
PS C:\Users\MSI 1\Downloads> ssh -i "dummy.pem" ubuntu@44.220.153.100
The authenticity of host '44.220.153.100 (44.220.153.100)' can't be established.
ED25519 key fingerprint is SHA256:6Xrw0t/5oUCJ+y1uhl0M10Ko3v8+WjWubZObJZ7yYc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.220.153.100' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)
```

```
ubuntu@ip-10-0-1-67:~$ ls
dummy.pem
ubuntu@ip-10-0-1-67:~$
```

Step 7: Secure the Key on the Bastion

- Once you are logged into the Bastion, you **must** change the permissions of the copied key. SSH will reject a key that is "too open" (readable by others).

```
# Run these on the Bastion Host
cd ~
chmod 400 your-key.pem
```

```
ubuntu@ip-10-0-1-67:~$ chmod 400 dummy.pem
ubuntu@ip-10-0-1-67:~$ ll
total 32
drwxr-x--- 4 ubuntu ubuntu 4096 Apr 24 10:21 ./
drwxr-xr-x 3 root   root   4096 Apr 24 10:17 ../
-rw-r--r-- 1 ubuntu ubuntu  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Mar 31  2024 .bashrc
drwx------ 2 ubuntu ubuntu 4096 Apr 24 10:19 .cache/
-rw-r--r-- 1 ubuntu ubuntu  807 Mar 31  2024 .profile
drwx------ 2 ubuntu ubuntu 4096 Apr 24 10:17 .ssh/
-rw-r--r-- 1 ubuntu ubuntu    0 Apr 24 10:19 .sudo_as_admin_successful
-r-------- 1 ubuntu ubuntu 1678 Apr 24 10:21 dummy.pem
ubuntu@ip-10-0-1-67:~$
```

Step 8: Jump to the Private Server

- Now that the key is sitting on the Bastion with the correct permissions, use it to connect to the internal IP of your private server.

```
# Run this on the Bastion Host
ssh -i "your-key.pem" ec2-user@PRIVATE_SERVER_PRIVATE_IP
```

```
ubuntu@ip-10-0-1-67:~$ ssh -i "dummy.pem" ubuntu@10.0.2.98
The authenticity of host '10.0.2.98 (10.0.2.98)' can't be established.
ED25519 key fingerprint is SHA256:6swq86QpNBYFF4PpoJ+e0rnO7TfxSJzBU8yVxau3Te8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.98' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
```

*Important Security Note

- By doing this, your private key is now stored on the Bastion's hard drive. If you are finished with your work, it is a best practice to **delete the key** from the Bastion to prevent a "lateral movement" risk if the Bastion is ever compromised:

  ```
  rm ~/your-key.pem
  ```

Conclusion

- By implementing this Bastion Host architecture, you have created a secure "chokepoint" that protects your private resources from direct internet exposure. This setup ensures that your sensitive servers remain isolated while still allowing authorized administrative access.
- **Quick Reference Checklist:**
  - **Access:** Bastion is in the **Public Subnet**; Server is in the **Private Subnet**.
  - **Security:** Only the Bastion's Security Group is allowed to SSH into the Private Server.
  - **Cleanup:** If you copied your `.pem` key to the Bastion, delete it immediately after use (`rm ~/keyname.pem`) to prevent security risks.
- **Next Steps:**
  - For production environments, consider exploring **AWS Systems Manager (SSM) Session Manager**, which allows you to "jump" into private instances without needing a public IP or managing SSH keys at all.